

Manipal School of Information Sciences

Linux Operating System Theory Assignment

Cybersecurity Log Analysis Using Linux Tools

SSH Brute-Force and Credential-Compromise Investigation

Student Name:	Ribhav Devprasad
Reg. No.:	26110690002
Date:	3 November 2026

Cybersecurity Log Analysis Using Linux Tools

1. Scenario and security significance

Scenario: investigating SSH authentication attacks against a Linux server (`labsrv01`) from its `auth.log`. The log covers two days and contains normal staff activity mixed with three attack patterns.

SSH is the main remote-administration service on Linux servers and is constantly probed from the Internet. A weak or reused password lets an attacker log in as a legitimate user, escalate with `sudo` and create persistence, and the authentication log is usually the first and sometimes the only evidence of this. The behaviours analysed map to MITRE ATT&CK techniques: **T1110.001** Password Guessing, **T1110.003** Password Spraying, **T1078** Valid Accounts and **T1136.001** Create Account (local).

2. How the log file was obtained

Safe simulation. The log was generated by a script, `generate_log.py` (Python 3, standard library only), run in an isolated Linux environment. The script writes syslog-style lines directly to a file from a fixed random seed (2026), so no real system was scanned, attacked or contacted and the same file is produced on every run. Public addresses come from the reserved documentation ranges `203.0.113.0/24` and `198.51.100.0/24` (RFC 5737, never routed on the Internet); internal users have private `192.168.1.x` addresses; user names are fictional; and the file contains no passwords, tokens or personal data.

```
$ python3 generate_log.py auth_simulated.log && wc -l auth_simulated.log
wrote 236 lines to auth_simulated.log
236 auth_simulated.log
```

```
$ python3 generate_log.py /tmp/check.log >/dev/null && cmp auth_simulated.log /tmp/check.log && echo
'identical: the simulation is repeatable'
identical: the simulation is repeatable
```

Planted activity (ground truth). Because the data is simulated, the correct answer is known and can be used to check the analysis:

ID	Activity	Source	Time	Intended meaning
A	62 failed logins in 3.5 minutes against `root` and common names	203.0.113.45	12 Oct 02:10	Fast brute-force attack, never succeeds
B	24 failed logins, 1 to 2 tries for each of 17 names, minutes apart	198.51.100.23	12 Oct 05:00 to 07:40	Slow password spraying
C	12 failures for `carol`, then a successful login, then `sudo` use and a new account	203.0.113.77	13 Oct 03:38	Attack that succeeds (compromise)
D	6 failures for `eve`, then success	192.168.1.35	12 Oct 14:20	Benign: internal user forgot password

E	Late-evening login by `bob`	192.168.1.21	12 Oct 23:32	Benign: off-hours but internal
F	5 connections that send no SSH banner	198.51.100.99	12 Oct 01:30 to 01:39	Scanner / banner grabbing
N	Normal logins, sessions, sudo and two typos	192.168.1.20 to . 23	08:00 to 18:00	Background noise

3. Structure of the log file

```
$ head -n 4 auth_simulated.log; echo ...
grep -n 'Accepted' auth_simulated.log | sed -n '1p'
grep -n 'sudo:' auth_simulated.log | sed -n '1p'
Oct 12 01:30:12 labsrv01 sshd[2601]: Did not receive identification string from 198.51.100.99 port 42414
Oct 12 01:30:14 labsrv01 sshd[1537]: Connection closed by 198.51.100.99 port 42414 [preauth]
Oct 12 01:33:02 labsrv01 sshd[4185]: Did not receive identification string from 198.51.100.99 port 37678
Oct 12 01:33:04 labsrv01 sshd[3946]: Connection closed by 198.51.100.99 port 37678 [preauth]
...
152:Oct 12 08:39:05 labsrv01 sshd[2676]: Accepted password for bob from 192.168.1.21 port 59531 ssh2
162:Oct 12 11:27:16 labsrv01 sudo: dave : TTY=pts/0 ; PWD=/home/dave ; USER=root ; COMMAND=/usr/bin/apt
update
```

Each line follows the standard syslog layout:

```
Oct 12 02:10:06 labsrv01 sshd[2231]: Failed password for root from 203.0.113.45 port 51822
ssh2
\_________/ \_______/ \______/ \____/
\_____________________________________________________/
 timestamp     time      host     process                          message
```

Field	Meaning and use in the analysis
Month, day, time	When the event happened. Used for time windows, per-hour counts and off-hours checks. The log has no year.
Hostname (`labsrv01`)	The machine that wrote the entry.
Process and PID (`sshd[2231]`, `sudo`, `useradd[..]`)	Which program logged the event; the PID groups lines of one connection.
Event text	`Failed password`, `Accepted password`, `Invalid user`, `session opened/closed`, `Did not receive identification string`, `sudo ... COMMAND=`, `new user:`.
`for <user>`	The account being used or targeted. `invalid user` means the account does not exist on the server.
`from <IP>` and `port`	Source address of the client and its ephemeral port. The IP is the key for grouping attackers.
`COMMAND=...` (sudo lines)	The privileged command executed. Shows what an attacker did after logging in.

4. Investigation questions

1. **Q1.** Which source IP addresses are performing brute-force attacks (many failures in a short time)?
2. **Q2.** Is any source trying many different usernames slowly, which a simple per-minute threshold would miss (password spraying)?
3. **Q3.** Did any attacker succeed in logging in, and what did the account do afterwards?
4. **Q4.** When do the attacks happen, and which accounts are targeted most?
5. **Q5.** Which suspicious-looking events are probably harmless (false positives)?

5. Tools used and design of the analysis

Tool	Role in the analysis
Power filters and pipelines (`grep`, `cut`, `sort`, `uniq -c`, `head`, `wc`)	Counting event types, ranking IPs and usernames (Q1, Q4)
sed	Extracting IPs and usernames from lines; tagging each line with an event class (`classify_events.sed`); masking IPs for sharing (`mask_ips.sed`)
awk	Per-IP statistics with a sliding time window (`detect_ips.awk`), hourly histogram (`hourly_failures.awk`), success-after-failure and post-login activity (`compromise_check.awk`)
Bash	Argument and file validation, running the stages in order, assembling one report (`analyse_auth_log.sh`)

The design is a two-step flow: **sed normalises** the raw text into tagged lines, then **awk analyses** the tagged lines by tag instead of re-matching long text patterns in every program. Bash joins everything and the filters give a quick cross-check of awk's totals.

6. The scripts

6.1 classify_events.sed

`classify_events.sed`

```
# classify_events.sed - prefix every auth-log line with an event tag
# Run with:  sed -E -f classify_events.sed auth.log
# The first matching rule tags the line and "b" jumps to the end of the script,
# so a line is never tagged twice (order matters: specific rules come first).
/Failed password for invalid user/    { s/^/[FAIL_INVALID_USER] /; b }
/Failed password/                     { s/^/[FAIL] /;              b }
/Accepted password/                   { s/^/[LOGIN_OK] /;          b }
/Invalid user/                        { s/^/[INVALID_USER] /;      b }
/COMMAND=/                            { s/^/[SUDO] /;              b }
/new user:/                           { s/^/[NEW_ACCOUNT] /;       b }
/Did not receive identification string/ { s/^/[SCAN] /;            b }
/session (opened|closed)/             { s/^/[SESSION] /;           b }
s/^/[OTHER] /
```

```
$ sed -E -f classify_events.sed auth_simulated.log | sed -n '1,3p;45,47p'     # sample of tagged lines
[SCAN] Oct 12 01:30:12 labsrv01 sshd[2601]: Did not receive identification string from 198.51.100.99 port
42414
[OTHER] Oct 12 01:30:14 labsrv01 sshd[1537]: Connection closed by 198.51.100.99 port 42414 [preauth]
[SCAN] Oct 12 01:33:02 labsrv01 sshd[4185]: Did not receive identification string from 198.51.100.99 port
37678
[FAIL] Oct 12 02:11:30 labsrv01 sshd[2835]: Failed password for root from 203.0.113.45 port 36745 ssh2
[FAIL] Oct 12 02:11:35 labsrv01 sshd[3841]: Failed password for root from 203.0.113.45 port 53038 ssh2
[OTHER] Oct 12 02:11:36 labsrv01 sshd[3841]: pam_unix(sshd:auth): authentication failure; logname= uid=0
euid=0 tty=ssh ruser= rhost=203.0.113.45  user=root
```

6.2 detect_ips.awk

detect_ips.awk

```
# detect_ips.awk - per-source-IP statistics and verdict
# Input : tagged log (output of classify_events.sed), so $1 is the event tag,
#         $2 $3 = month day, $4 = time, then host, process, message
# Output: one line per IP:  IP SCOPE FAILS USERS MAX_IN_WINDOW FIRST LAST LOGINS_OK VERDICT
# Tunable thresholds (override with  awk -v BURST_MIN=8 ... ):
#   WINDOW     sliding window length in seconds          (default 300 = 5 min)
#   BURST_MIN  failures inside one window => brute force (default 10)
#   SPRAY_MIN  distinct usernames tried => password spraying (default 5)
#   TOTAL_MIN  total failures from one IP => persistent attacker (default 20)
#   WATCH_MIN  failures inside one window => watch-list  (default 5)

function epoch(day, hms,   t) { split(hms, t, ":"); return day * 86400 + t[1] * 3600 + t[2] *
60 + t[3] }
function is_private(ip) { return ip ~ /^(10\.|192\.168\.|172\.(1[6-9]|2[0-9]|3[01])\.)/ }

BEGIN {
    if (WINDOW == "")    WINDOW = 300
    if (BURST_MIN == "") BURST_MIN = 10
    if (SPRAY_MIN == "") SPRAY_MIN = 5
    if (TOTAL_MIN == "") TOTAL_MIN = 20
    if (WATCH_MIN == "") WATCH_MIN = 5
}

$1 == "[FAIL]" || $1 == "[FAIL_INVALID_USER]" {
    ip = ""; user = ""
    for (i = 5; i <= NF; i++) {
        if ($i == "for" && user == "") { user = $(i + 1); if (user == "invalid") user = $(i +
3) }
        if ($i == "from") ip = $(i + 1)
    }
    if (ip == "") next
    ts = epoch($3, $4)
    fails[ip]++
    if (!((ip, user) in seen)) { seen[ip, user] = 1; users[ip]++ }
    n[ip]++; stamp[ip, n[ip]] = ts                    # sliding window over this IP's
failures
    while (ts - stamp[ip, head[ip] + 1] > WINDOW) head[ip]++
    if (n[ip] - head[ip] > maxwin[ip]) maxwin[ip] = n[ip] - head[ip]
    if (!(ip in first)) first[ip] = $2 $3 "-" $4
    last[ip] = $2 $3 "-" $4
}

$1 == "[LOGIN_OK]" {
    for (i = 5; i <= NF; i++) if ($i == "from") ok[$(i + 1)]++
}

END {
    for (ip in fails) {
        if (maxwin[ip] >= BURST_MIN)        v = "BRUTE_FORCE"
        else if (users[ip] >= SPRAY_MIN)    v = "PASSWORD_SPRAYING"
        else if (fails[ip] >= TOTAL_MIN)    v = "PERSISTENT"
        else if (maxwin[ip] >= WATCH_MIN)   v = "WATCH"
        else                                v = "normal"
        printf "%s %s %d %d %d %s %s %d %s\n", ip, (is_private(ip) ? "internal" :
```

```
"external"),
               fails[ip], users[ip], maxwin[ip], first[ip], last[ip], ok[ip] + 0, v
    }
}
```

6.3 hourly_failures.awk

hourly_failures.awk

```
# hourly_failures.awk - failed logins per hour as a text bar chart (1 '#' = 2 failures)
# Input: tagged log. Output order follows the log (chronological).
$1 == "[FAIL]" || $1 == "[FAIL_INVALID_USER]" {
    split($4, t, ":")
    key = $2 " " $3 " " t[1] ":00"
    if (!(key in cnt)) order[++k] = key
    cnt[key]++
}
END {
    for (i = 1; i <= k; i++) {
        bar = ""
        for (j = 0; j < int((cnt[order[i]] + 1) / 2); j++) bar = bar "#"
        printf "%s  %3d  %s\n", order[i], cnt[order[i]], bar
    }
}
```

6.4 compromise_check.awk

compromise_check.awk

```
# compromise_check.awk - look for a successful login that follows many failures
# Input: tagged log in time order.
# Rules:
#   R1  [LOGIN_OK] from an IP that had >= MIN_FAILS failures before it
#       -> CRITICAL if the IP is external, REVIEW if internal (possible forgotten password)
#   R2  [LOGIN_OK] from an external IP between 00:00 and 05:59 -> OFF_HOURS_EXTERNAL
#   R3  For logins flagged by R1, list sudo commands and new accounts within FOLLOW seconds
function epoch(day, hms,   t) { split(hms, t, ":"); return day * 86400 + t[1] * 3600 + t[2] *
60 + t[3] }
function is_private(ip) { return ip ~ /^(10\.|192\.168\.|172\.(1[6-9]|2[0-9]|3[01])\.)/ }
BEGIN { if (MIN_FAILS == "") MIN_FAILS = 5; if (FOLLOW == "") FOLLOW = 1800 }

$1 == "[FAIL]" || $1 == "[FAIL_INVALID_USER]" {
    for (i = 5; i <= NF; i++) if ($i == "from") pf[$(i + 1)]++
}
$1 == "[LOGIN_OK]" {
    user = ""; ip = ""
    for (i = 5; i <= NF; i++) { if ($i == "for") user = $(i + 1); if ($i == "from") ip = $(i
+ 1) }
    ts = epoch($3, $4); split($4, t, ":")
    if (pf[ip] >= MIN_FAILS) {
        sev = is_private(ip) ? "REVIEW  " : "CRITICAL"
        printf "[%s] %s %s %s  login for '%s' from %s after %d failed attempts\n", sev, $2,
$3, $4, user, ip, pf[ip]
        if (!is_private(ip)) { watch[user] = ts; watchip[user] = ip }
        pf[ip] = 0
    } else if (t[1] + 0 < 6 && !is_private(ip)) {
        printf "[OFF_HOURS_EXTERNAL] %s %s %s  login for '%s' from %s\n", $2, $3, $4, user,
ip
    } else if (t[1] + 0 >= 22 || t[1] + 0 < 6) {
        printf "[INFO    ] %s %s %s  off-hours login for '%s' from internal address %s (not
escalated)\n", $2, $3, $4, user, ip
    }
}
$1 == "[SUDO]" {
    u = $7
    if ((u in watch) && epoch($3, $4) - watch[u] <= FOLLOW) {
```

```
        match($0, /COMMAND=.*/)
        printf "    -> %s  sudo by '%s': %s\n", $4, u, substr($0, RSTART + 8, RLENGTH - 8)
    }
}
$1 == "[NEW_ACCOUNT]" {
    for (u in watch) if (epoch($3, $4) - watch[u] <= FOLLOW) {
        match($0, /name=[^,]*/)
        printf "    -> %s  NEW ACCOUNT created: %s\n", $4, substr($0, RSTART + 5, RLENGTH -
5)
        break
    }
}
```

6.5 mask_ips.sed

mask_ips.sed

```
# mask_ips.sed - hide the last octet of every IPv4 address (for sharing a report)
# Run with:  sed -E -f mask_ips.sed report.txt
s/([0-9]{1,3}\.[0-9]{1,3}\.[0-9]{1,3})\.[0-9]{1,3}/\1.xxx/g
```

6.6 analyse_auth_log.sh (main Bash script)

analyse_auth_log.sh

```
#!/bin/bash
# analyse_auth_log.sh - SSH authentication-log analysis using grep, sort/uniq, sed and awk
#
# Usage:   ./analyse_auth_log.sh [-m] [-o OUTDIR] LOGFILE
#   -m        mask the last octet of every IP address in the final report (safe to share)
#   -o DIR    write results into DIR (default: ./output)
#
# Pipeline overview
#   1. grep / grep -c            quick counts of each event type
#   2. sed | sort | uniq -c      top attacking IPs and targeted usernames
#   3. sed -f classify_events.sed   tag every line with an event class  -> tagged_events.log
#   4. awk -f detect_ips.awk     per-IP statistics + verdict            -> ip_summary.txt
#   5. awk -f hourly_failures.awk   failures per hour (bar chart)       ->
hourly_failures.txt
#      (also: scanner sources via grep | sed | sort | uniq -c)
#   6. awk -f compromise_check.awk  success-after-failures and what followed ->
compromise_findings.txt
#   7. everything is assembled into security_report.txt

set -u
SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
OUTDIR="output"
MASK=0

usage() { echo "Usage: $0 [-m] [-o OUTDIR] LOGFILE" >&2; exit 1; }

while getopts ":mo:" opt; do
    case "$opt" in
        m) MASK=1 ;;
        o) OUTDIR="$OPTARG" ;;
        *) usage ;;
    esac
done
shift $((OPTIND - 1))
[ $# -eq 1 ] || usage
LOG="$1"

# ---- input validation
[ -f "$LOG" ] || { echo "Error: '$LOG' is not a file." >&2; exit 2; }
[ -r "$LOG" ] || { echo "Error: '$LOG' is not readable." >&2; exit 3; }
[ -s "$LOG" ] || { echo "Error: '$LOG' is empty." >&2; exit 4; }
```

```
for f in classify_events.sed mask_ips.sed detect_ips.awk hourly_failures.awk
compromise_check.awk; do
    [ -f "$SCRIPT_DIR/$f" ] || { echo "Error: helper script $f not found next to this
script." >&2; exit 5; }
done
mkdir -p "$OUTDIR" || exit 6

TAGGED="$OUTDIR/tagged_events.log"
SUMMARY="$OUTDIR/ip_summary.txt"
HOURLY="$OUTDIR/hourly_failures.txt"
FINDINGS="$OUTDIR/compromise_findings.txt"
REPORT="$OUTDIR/security_report.txt"

# ---- step 3: classify with sed (CRLF removed first so $ anchors and fields behave)
tr -d '\r' < "$LOG" | sed -E -f "$SCRIPT_DIR/classify_events.sed" > "$TAGGED"

# ---- step 4: per-IP statistics (awk), sorted by failures, most first
{
    printf "%-16s %-9s %6s %6s %9s %-15s %-15s %5s  %s\n" IP SCOPE FAILS USERS MAX/5MIN FIRST
LAST OK VERDICT
    awk -f "$SCRIPT_DIR/detect_ips.awk" "$TAGGED" | sort -k3,3nr |
        awk '{ printf "%-16s %-9s %6d %6d %9d %-15s %-15s %5d  %s\n",
$1,$2,$3,$4,$5,$6,$7,$8,$9 }'
} > "$SUMMARY"

# ---- step 5 and 6
awk -f "$SCRIPT_DIR/hourly_failures.awk" "$TAGGED" > "$HOURLY"
awk -f "$SCRIPT_DIR/compromise_check.awk" "$TAGGED" > "$FINDINGS"

# ---- step 1 and 2: counts and top lists
count() { grep -c -E "$1" "$LOG"; }

{
    echo "============================================================"
    echo " SSH AUTHENTICATION LOG ANALYSIS REPORT"
    echo " Log file : $LOG"
    echo " Lines    : $(wc -l < "$LOG")"
    echo " Period   : $(head -n 1 "$LOG" | cut -d' ' -f1-3)  to  $(tail -n 1 "$LOG" | cut -d'
' -f1-3)"
    echo "============================================================"
    echo
    echo "[1] EVENT COUNTS"
    printf "  %-34s %5d\n" "Failed password attempts"        "$(count 'Failed password')"
    printf "  %-34s %5d\n" "  of which for non-existent users" "$(count 'Failed password for
invalid user')"
    printf "  %-34s %5d\n" "Accepted logins"                 "$(count 'Accepted password')"
    printf "  %-34s %5d\n" "sudo commands"                   "$(count 'COMMAND=')"
    printf "  %-34s %5d\n" "New accounts created"            "$(count 'new user:')"
    printf "  %-34s %5d\n" "Scanner banner-grab events"      "$(count 'Did not receive
identification string')"
    echo
    echo "[2] TOP 5 SOURCE IPs BY FAILED LOGINS (grep | sed | sort | uniq -c | sort -nr |
head)"
    grep 'Failed password' "$LOG" | sed -E 's/.* from ([0-9.]+) port.*/\1/' | sort | uniq -c
| sort -nr | head -n 5 |
        awk '{ printf "  %5d  %s\n", $1, $2 }'
    echo
    echo "[3] TOP 8 TARGETED USERNAMES"
    grep 'Failed password' "$LOG" | sed -nE 's/.*Failed password for (invalid user )?([^ ]+)
from .*/\2/p' |
        sort | uniq -c | sort -nr | head -n 8 | awk '{ printf "  %5d  %s\n", $1, $2 }'
    echo
    echo "[4] SCANNER-LIKE SOURCES (connections that never attempt to authenticate)"
    grep 'Did not receive identification string' "$LOG" | sed -E 's/.* from ([0-9.]+) port.*/
\1/' |
        sort | uniq -c | sort -nr | awk '{ printf "  %5d  %s\n", $1, $2 }'
    echo
    echo "[5] PER-IP ANALYSIS (verdict thresholds explained in the report)"
    sed 's/^/  /' "$SUMMARY"
    echo
```

```
    echo "[6] FAILED LOGINS PER HOUR  (# = 2 failures)"
    sed 's/^/  /' "$HOURLY"
    echo
    echo "[7] SUCCESSFUL LOGINS FOLLOWING FAILURES, AND WHAT HAPPENED NEXT"
    if [ -s "$FINDINGS" ]; then sed 's/^/  /' "$FINDINGS"; else echo "  none"; fi
    echo
    echo "[8] ALERT SUMMARY"
    for v in BRUTE_FORCE PASSWORD_SPRAYING PERSISTENT WATCH; do
        ips=$(awk -v v="$v" 'NR > 1 && $NF == v { printf "%s ", $1 }' "$SUMMARY")
        [ -n "$ips" ] && echo "  [ALERT] $v : $ips"
    done
    scan=$(grep 'Did not receive identification string' "$LOG" | sed -E 's/.* from ([0-9.]+)
port.*/\1/' |
           sort | uniq -c | awk '$1 >= 3 { printf "%s ", $2 }')
    [ -n "$scan" ] && echo "  [ALERT] SCANNER : $scan"
    grep -E '^\[(CRITICAL|OFF_HOURS_EXTERNAL)' "$FINDINGS" | sed 's/^/  [ALERT] /'
    echo
    echo "END OF REPORT"
} > "$REPORT"

# ---- optional masking of addresses for sharing
if [ "$MASK" -eq 1 ]; then
    sed -E -i -f "$SCRIPT_DIR/mask_ips.sed" "$REPORT"
fi

cat "$REPORT"
echo
echo "Files written to $OUTDIR/: tagged_events.log ip_summary.txt hourly_failures.txt
compromise_findings.txt security_report.txt" >&2
```

7. Purpose of each command

Command or construct	Purpose
`set -u`	Treat use of an undefined variable as an error
`SCRIPT_DIR="$(cd "$(dirname "$ {BASH_SOURCE[0]}")" && pwd)"`	Find the folder of the script so the sed and awk helper files are found from any directory
`getopts ":mo:"`, `shift $((OPTIND - 1))`	Parse the options `-m` and `-o DIR`, then leave only the log file name in `$1`
`[ -f ] / [ -r ] / [ -s ]` with `exit N`	Validate the input and stop with a specific exit code and message
`tr -d '\r' < "$LOG"`	Remove Windows carriage returns so fields and anchors behave
`sed -E -f classify_events.sed`	Tag each line with an event class; `b` jumps to the end so a line is tagged once
`awk -f detect_ips.awk`	Build per-IP counters in associative arrays and apply the verdict rules in `END`
`sort -k3,3nr`	Order the per-IP table by number of failures, highest first
`awk '{ printf "%-16s ..." }'`	Align the columns of the table
`grep -c -E`	Count lines of an event type (used for the totals and as a cross-check of awk)

`grep | sed -E 's/.* from ([0-9.]+) port.*/\1/' | sort | uniq -c | sort -nr | head`	Select failures, extract the IP with a capture group, group equal IPs, count them, rank them and keep the top ones. `sort` before `uniq -c` is required because `uniq` counts only adjacent duplicates
`sed -nE 's/...(invalid user )?([^ ]+) from .*/\2/p'`	Extract the targeted username; `-n` with `p` prints only lines where the substitution matched
`awk -v v="$v" 'NR > 1 && $NF == v ...'`	Pass a shell variable into awk, skip the header and select rows whose last field is the verdict
`{ ...; } > "$REPORT"`	Group several commands so all their output goes into one file
`sed -E -i -f mask_ips.sed`	Edit the finished report in place to hide the last octet of every IP when `-m` is used

Inside the awk programs: `split($4,t,":")` splits the time into hour, minute, second; `epoch()` turns day plus time into seconds so a 5-minute window can be computed; `stamp[ip,n]` and `head[ip]` implement a **sliding window** by moving a start pointer forward while the oldest failure is older than `WINDOW` seconds; `seen[ip,user]` counts each distinct (IP, username) pair once; `is_private()` uses a regular expression for the RFC 1918 ranges; and `match()` with `RSTART`/`RLENGTH` with `substr()` extracts the sudo command text.

8. Output and findings

```
$ ./analyse_auth_log.sh auth_simulated.log 2>&1
================================================================
 SSH AUTHENTICATION LOG ANALYSIS REPORT
 Log file : auth_simulated.log
 Lines    : 236
 Period   : Oct 12 01:30:12  to  Oct 13 19:12:30
================================================================

[1] EVENT COUNTS
  Failed password attempts            107
    of which for non-existent users    42
  Accepted logins                      19
  sudo commands                         6
  New accounts created                  1
  Scanner banner-grab events            5

[2] TOP 5 SOURCE IPs BY FAILED LOGINS (grep | sed | sort | uniq -c | sort -nr | head)
     62  203.0.113.45
     24  198.51.100.23
     12  203.0.113.77
      6  192.168.1.35
      2  192.168.1.21

[3] TOP 8 TARGETED USERNAMES
     42  root
     12  carol
      9  test
      6  eve
      5  admin
      4  ubuntu
      4  postgres
      3  user

[4] SCANNER-LIKE SOURCES (connections that never attempt to authenticate)
      5  198.51.100.99

[5] PER-IP ANALYSIS (verdict thresholds explained in the report)
  IP              SCOPE      FAILS  USERS  MAX/5MIN FIRST           LAST              OK  VERDICT
  203.0.113.45    external      62      7        62 Oct12-02:10:06  Oct12-02:13:41      0  BRUTE_FORCE
  198.51.100.23   external      24     17         2 Oct12-05:00:18  Oct12-07:40:54      0  PASSWORD_SPRAYING
```

```
  203.0.113.77     external      12      1        12 Oct13-03:38:11  Oct13-03:39:34      1  BRUTE_FORCE
  192.168.1.35     internal       6      1         6 Oct12-14:20:06  Oct12-14:21:16      1  WATCH
  192.168.1.21     internal       2      1         2 Oct12-10:05:13  Oct12-10:05:32      7  normal
  192.168.1.23     internal       1      1         1 Oct13-09:14:04  Oct13-09:14:04      4  normal

[6] FAILED LOGINS PER HOUR  (# = 2 failures)
  Oct 12 02:00   62  ##############################
  Oct 12 05:00    9  #####
  Oct 12 06:00    9  #####
  Oct 12 07:00    6  ###
  Oct 12 10:00    2  #
  Oct 12 14:00    6  ###
  Oct 13 03:00   12  ######
  Oct 13 09:00    1  #

[7] SUCCESSFUL LOGINS FOLLOWING FAILURES, AND WHAT HAPPENED NEXT
  [REVIEW  ] Oct 12 14:22:10  login for 'eve' from 192.168.1.35 after 6 failed attempts
  [INFO    ] Oct 12 23:32:41  off-hours login for 'bob' from internal address 192.168.1.21 (not escalated)
  [CRITICAL] Oct 13 03:39:47  login for 'carol' from 203.0.113.77 after 12 failed attempts
      -> 03:41:12  sudo by 'carol': /bin/bash
      -> 03:42:25  sudo by 'carol': /bin/cat /etc/shadow
      -> 03:43:02  sudo by 'carol': /usr/sbin/useradd -m svc_backup
      -> 03:43:03  NEW ACCOUNT created: svc_backup

[8] ALERT SUMMARY
  [ALERT] BRUTE_FORCE : 203.0.113.45 203.0.113.77
  [ALERT] PASSWORD_SPRAYING : 198.51.100.23
  [ALERT] WATCH : 192.168.1.35
  [ALERT] SCANNER : 198.51.100.99
  [ALERT] [CRITICAL] Oct 13 03:39:47  login for 'carol' from 203.0.113.77 after 12 failed attempts

END OF REPORT

Files written to output/: tagged_events.log ip_summary.txt hourly_failures.txt compromise_findings.txt
security_report.txt
```

Answers to the investigation questions

1. **Q1: Brute force.** `203.0.113.45` made 62 failed attempts, all inside one 5-minute window (02:10 to 02:13 on 12 Oct), against 7 names with `root` as the main target (42 of the failures). `203.0.113.77` made 12 failures against `carol` in 84 seconds. Both exceed the burst threshold.
2. **Q2: Spraying.** `198.51.100.23` tried 17 different usernames with only 24 attempts over about 2 hours 40 minutes. Its busiest 5-minute window held just 2 failures, so the burst rule cannot see it; the distinct-username rule catches it (`PASSWORD_SPRAYING`). This is the stealthy pattern designed to stay under lockout and rate limits.
3. **Q3: Compromise.** The **critical finding**: `203.0.113.77` logged in as `carol` at 03:39:47 on 13 Oct, immediately after 12 failures, from an external address at 03:39 (off-hours). Within four minutes the account ran `sudo /bin/bash`, `sudo cat /etc/shadow` (reading password hashes) and `sudo useradd -m svc_backup`, and a new account `svc_backup` was created. This sequence of guessing, access, privilege use, credential access and a new account for persistence is a textbook account compromise. `203.0.113.45` and `198.51.100.23` have **0 successful logins**, so those attacks failed.
4. **Q4: Time and targets.** The hourly chart shows activity concentrated between 02:00 and 07:00 and at 03:00 on the second day, when staff are not working; `root` is the most targeted account, followed by `carol` (the compromised account) and common service names such as `test`, `admin`, `ubuntu` and `postgres`.
5. **Q5: False positives.** See section 10. In this run `192.168.1.35` (`eve`) is flagged `WATCH` and `REVIEW`, and `bob`'s 23:32 login is recorded as `INFO` only.

Check against the ground truth

Planted	Detected as	Result

A: fast brute force, 203.0.113.45	BRUTE_FORCE	Correct
B: slow spraying, 198.51.100.23	PASSWORD_SPRAYING	Correct (not caught by the burst rule alone)
C: successful compromise, 203.0.113.77	BRUTE_FORCE and [CRITICAL] success-after-failures with sudo and new-account evidence	Correct
D: eve forgot password	WATCH and [REVIEW]	False positive, correctly given lower severity because the source is internal
E: bob late login	[INFO] only	Correctly not escalated
F: scanner, 198.51.100.99	SCANNER	Correct (has no failed passwords, so only the banner-grab count finds it)

```
$ ./analyse_auth_log.sh -m auth_simulated.log     # -m masks the last octet of IPs, shown for section [2]
[2] TOP 5 SOURCE IPs BY FAILED LOGINS (grep | sed | sort | uniq -c | sort -nr | head)
     62  203.0.113.xxx
     24  198.51.100.xxx
     12  203.0.113.xxx
      6  192.168.1.xxx
      2  192.168.1.xxx

[3] TOP 8 TARGETED USERNAMES
```

```
$ ./analyse_auth_log.sh; echo "exit status: $?"; ./analyse_auth_log.sh no_such_file.log; echo "exit status:
$?"
Usage: ./analyse_auth_log.sh [-m] [-o OUTDIR] LOGFILE
exit status: 1
Error: 'no_such_file.log' is not a file.
exit status: 2
```

```
$ awk -v BURST_MIN=15 -f detect_ips.awk output/tagged_events.log | sort -k3,3nr | awk '{print $1, $3, $5,
$9}'   # change a threshold without editing the script
203.0.113.45 62 62 BRUTE_FORCE
198.51.100.23 24 2 PASSWORD_SPRAYING
203.0.113.77 12 12 WATCH
192.168.1.35 6 6 WATCH
192.168.1.21 2 2 normal
192.168.1.23 1 1 normal
```

The last run shows the thresholds are parameters: raising `BURST_MIN` to 15 stops `203.0.113.77` (12 failures) being called a brute-force attack, and it becomes `PERSISTENT` or `WATCH` instead, which shows why the threshold choice matters (section 9).

9. Justification of thresholds

Rule	Value	Justification

Brute force: failures from one IP inside a sliding window	**≥ 10 in 300 s**	A person mistyping a password fails 1 to 3 times, and OpenSSH disconnects a connection after 6 tries by default (`MaxAuthTries`). Ten failures in five minutes means roughly one every 30 seconds or faster, which is automated. In the data the attacks reached 62 and 12, while the largest benign burst was 6.
Password spraying: distinct usernames from one IP	**≥ 5**	One person normally uses 1 or 2 accounts. Trying five or more different names, including non-existent ones, indicates enumeration. This catches attacks that deliberately stay under the failure-rate threshold.
Persistent attacker: total failures from one IP	**≥ 20**	A catch-all for slow attackers that vary names and timing but keep returning during the log period.
Watch list	**≥ 5 in 300 s**	Low severity: more than a typo, but not yet a pattern. Prompts a human to look.
Success after failures	**≥ 5 prior failures from the same IP**	A login that follows repeated failures from the same source is the signature of a successful guess. Severity is **CRITICAL** for an external address and only **REVIEW** for an internal one.
Off-hours external login	**00:00 to 05:59, non-private IP**	Staff in this simulation work 08:00 to 18:00; an external login in the small hours is unusual. Internal off-hours logins are only logged as INFO to limit noise.
Post-login follow window	**1800 s**	Attacker actions usually come soon after access, so sudo commands and new accounts within 30 minutes of a suspicious login are linked to it.
Scanner	**≥ 3** connections with no banner	A real client always sends an identification string; repeated connections without one are probes.

These values are tuned to this log and the OpenSSH defaults. In production they should be tuned against a baseline of that server's normal behaviour.

10. False positives and limitations

Possible false positives

- **Forgotten or mistyped password**: the internal user `eve` (6 failures then success) matches the success-after-failures rule. Treating internal sources as REVIEW instead of CRITICAL reduces the noise, but a human still has to confirm.
- **NAT, proxies, VPNs and shared gateways**: many users behind one public IP can add up to a burst, and one user's failures can be attributed to the whole group.
- **Automation with an expired credential**: a backup job, monitoring system or CI pipeline retrying an old password produces a steady stream of failures that looks like brute force.
- **Legitimate off-hours work**: administrators in other time zones or on call can trigger the off-hours rule; `bob`'s late login is an example that is only logged as INFO because it is internal.
- **Vulnerability scanners run by the organisation** look like attackers and produce the scanner and brute-force verdicts.

Limitations

- **Simulated data.** The log is clean and the patterns are deliberately clear. Real logs are larger, noisier and have more varied formats, so the thresholds would need tuning and the results here overstate real-world accuracy.
- **Only password authentication is covered.** Public-key logins, Kerberos or `su` are not analysed, and a stolen SSH key would leave no failures to detect.
- **Single log source and single host.** A distributed attack that uses many IPs with a few attempts each stays below every per-IP threshold, and without firewall or network logs there is no confirmation of the real source.
- **A successful login with no earlier failures** (stolen or reused valid credentials) is not detected except by the off-hours rule.
- **Time handling is simplified.** The log has no year or time zone and the epoch calculation assumes one month, so a log spanning a month end would need a better date conversion. Log rotation can also split one attack across files.
- **Logs can be tampered with.** An attacker with root access can edit or delete log entries, so evidence should be copied to a central log server.
- **Spoofing and shared IPs.** The source IP in an SSH log is a real TCP peer (it cannot be spoofed for a completed connection) but may be a proxy or compromised host rather than the attacker's own machine.
- **Detection, not prevention.** The script reports after the fact. Real mitigations are key-based authentication, `PermitRootLogin no`, rate limiting or `fail2ban`, and resetting the `carol` credentials and removing `svc_backup` in this scenario.

Submission contents and execution

Item	File
Assignment report	this document
Log file	`auth_simulated.log` (generated by `generate_log.py`)
Bash script	`analyse_auth_log.sh`
sed and awk script files	`classify_events.sed`, `mask_ips.sed`, `detect_ips.awk`, `hourly_failures.awk`, `compromise_check.awk`
Generated output	`output/` folder (`security_report.txt`, `ip_summary.txt`, `hourly_failures.txt`, `compromise_findings.txt`, `tagged_events.log`)
README	`README.md` (execution instructions and dataset details)
Dataset source	Simulated (no download); described in section 2 and the README

To run: `chmod +x analyse_auth_log.sh && ./analyse_auth_log.sh auth_simulated.log`

Resources and assistance used

- Linux manual pages for `grep`, `sed`, `awk`, `sort`, `uniq` and `bash`; the GNU sed and gawk manuals; course material on power filters, Bash, sed and awk.
- MITRE ATT&CK technique pages T1110, T1078 and T1136 for terminology.
- OpenSSH documentation for the default `MaxAuthTries` value used to justify a threshold.
- **AI assistance:** Claude (Anthropic) was used to help design and write the simulation script, the Bash, sed and awk scripts and the text of this report. The scripts were executed and their output reviewed before submission.